

```

Output
Code
[Satya]~/Article-IntegralPromotion/Codes
gcc test_2.c
[Satya]~/Article-IntegralPromotion/Codes
./a.out
The value of x: FFFFFFFF
The value of y: FF
[Satya]~/Article-IntegralPromotion/Codes

```

Figure 2: Output of Code-2

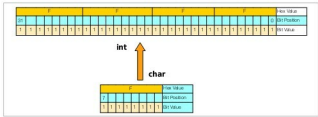


Figure 3: Pictorial representation of char promoted to int type

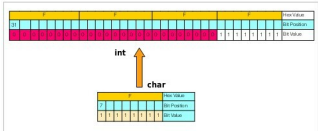


Figure 4: Pictorial representation of unsigned char promoted to int type

In order to understand why the output is ‘unequal’, let us look at one more simple example, Code-2:

```

1 #include <stdio.h>
2
3 int main()
4 {
5     signed char x = 0xFF;
6     unsigned char y = 0xFF;
7
8     printf("The value of x: %X\n", x);
9     printf("The value of y: %X\n", y);
10
11     return 0;
12 }

```

Code-2: Code to print the values of x and y

Since x is declared as the *char* data type, one byte of memory will be allocated and 0xFF is the value stored in it. So, printing the value of x using the ‘%X’ format specifier results in integer type promotion. Also, since the MSB bit is set in this case, when it is promoted, all the bits in the remaining three bytes of the *int* type will be filled with the value of the MSB bit. Hence, all the bits will be filled with ‘1’, because the MSB bit is ‘1’ in the ‘x’ variable as shown in Figure 3.

Also, since ‘y’ is declared as the *unsigned char* data type, one

byte of memory will be allocated and 0xFF is the value stored in it. So, printing the value of ‘x’ using the ‘%X’ format specifier will result in integer type promotion. Since the MSB is not reserved for storing the sign of the number when it is promoted, all the bits in the remaining three bytes of the *int* type will be filled with zeros, irrespective of whether MSB of lower rank data type is SET or CLEAR, as shown in Figure 4.

So from the explanation given for Code-2, it is clear that *char* is promoted to *int* data type. Now going back to Code-1, when two variables are compared in the ‘if’ statement, integer type promotion takes place. As a result, the value of ‘x’ is converted to 0xFFFFFFFF since ‘x’ is a *signed char*, and the value of ‘y’ is converted to 0x000000FF because ‘y’ is *unsigned char*. After implicit type promotion, the values are compared and found ‘unequal’. For more clarity, refer to Figure 5.

Let us dig a little deeper by obtaining the assembly code for Code-1 and analysing the disassembled code as shown in Figure 6, the explanation for which is given in Table 1.

| Assembly instruction          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>movb \$0xff,0x1e(%esp)</b> | A byte(0xff) is moved to 0x1e pointed by <i>esp</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>movb \$0xff,0x1f(%esp)</b> | A byte(0xff) is moved to 0x1f pointed by <i>esp</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>movsbl 0x1e(%esp),%edx</b> | This instruction copies the source operand to the destination, and pads the remaining bits not provided by source with the sign bit (MSB) of the source. OR sign-extends a single byte, and copies it into a double-word destination. Assuming the machine is x86, the MOVSBIL instruction extends a byte (8 bits), representing a signed number, to a 32-bit signed/unsigned number. The remaining 24 bits are zeros or ones depending on the sign, so that the two's complement value remains. So here, after moving <i>%esp</i> = 0xFF to <i>%edx</i> , the value will be <i>%edx</i> = 0xFFFFFFFF |
| <b>movzbl 0x1f(%esp),%eax</b> | This instruction expands a single byte to 32 bits with 24 leading zeros, and copies it into a double-word destination. Assuming the machine is x86, the MOVZBL instruction extends a byte (8 bits) representing an unsigned number to a 32-bit signed/unsigned number. The remaining 24 bits are zeros. So here, after moving <i>%esp</i> = 0xFF to <i>%eax</i> , the value will be <i>%eax</i> = 0x000000FF                                                                                                                                                                                          |